

Machine Code & AArch64 Instruction Encoding

Instruction words, encoding fields, immediates, memory operations, and branch targets in AArch64

Dr. J. Burns

Topic 7 Outline

1. Machine Code and 32-bit Instruction Words

- Assembly text, instruction words, bit fields, and byte order.

2. AArch64 Encoding Families

- How high-order bits select data-processing, load/store, and branch formats.

3. Data-Processing Encodings

- Register operands, immediate operands, and worked examples.

4. Memory and Branch Encodings

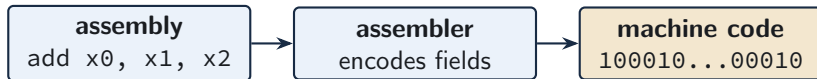
- Load/store offsets, PC-relative addressing, unconditional branches, and conditional branches.

Topic 7

Machine Code and 32-bit Instruction Words

From Assembly to Machine Code

- A processor repeatedly **fetches**, **decodes**, and **executes** instructions.
- During fetch–decode–execute, the processor fetches a 32-bit instruction word from memory and interprets its bit fields.
- This topic starts with the earlier step: how an AArch64 assembly instruction is encoded as that fixed **32-bit** machine instruction.



AArch64 vs. AArch32 Encoding

Shared 32-bit Structure

- Instructions are encoded as fields inside a 32-bit word.
- Different instruction classes use different field layouts.
- A small number of high-level formats helps the decoder.

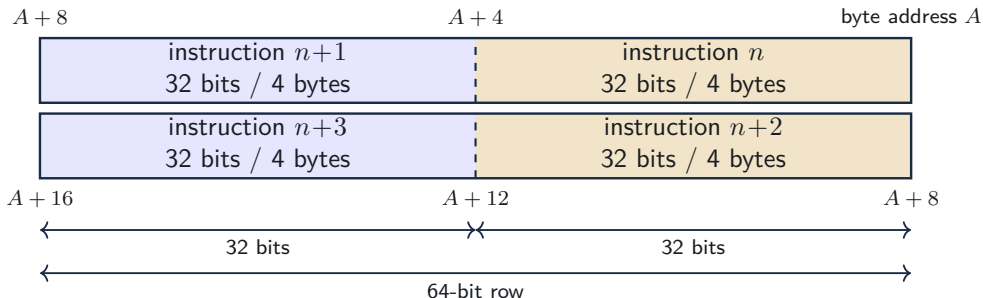
AArch64 Encoding

- There is no 4-bit condition field in every instruction.
- Register fields are usually 5 bits because there are 32 integer registers.
- The high-order bits identify a broader encoding family.

Machine code is a structured 32-bit word whose fields the processor decodes.

AArch64 Instruction Word Basics

Each AArch64 instruction is 32 bits, or 4 bytes. Four consecutive instructions therefore occupy two 64-bit memory rows.



Instruction Boundaries

Each instruction starts 4 bytes after the previous one. A 64-bit memory row holds two AArch64 instructions.

How the Decoder Starts

- AArch64 has several major encoding families.
- The decoder first checks high-order opcode bits to select the family.
- After the family is known, the remaining fields can be interpreted.

Examples of instruction families

- Data processing: arithmetic, logical, moves, shifts, and compares.
- Load/store: memory access through base registers and offsets.
- Branch: unconditional branches, calls, returns, and conditional branches.

Identify the broad instruction family first, then read the fields that belong to that family.

Topic 7

Common Encoding Fields and Conditional Patterns

Common AArch64 Encoding Prefixes

AArch64 has no single universal opcode field. Fixed high-order bits identify the instruction form, and the remaining bits encode operands.

Instruction form	Fixed high-order bits	Remaining fields
add x, x, x	10001011000	Rm, shift, Rn, Rd
sub x, x, x	11001011000	Rm, shift, Rn, Rd
add x, x, #imm	10010001	shift, imm12, Rn, Rd
sub x, x, #imm	11010001	shift, imm12, Rn, Rd
ldr x, [x, #imm]	1111100101	imm12, Rn, Rt
str x, [x, #imm]	1111100100	imm12, Rn, Rt
b label	000101	imm26
bl label	100101	imm26
b.cond label	01010100	imm19, cond

Reading the table: these prefixes identify the form; they are not complete instructions. Operand fields fill the remaining bits.

Conditional Execution in AArch64

- AArch32 allowed a condition code on most instructions; AArch64 does not.
- AArch64 uses conditional branches and selected conditional operations instead.
- Conditions are evaluated from the NZCV flags set by earlier arithmetic or compare instructions.

Example

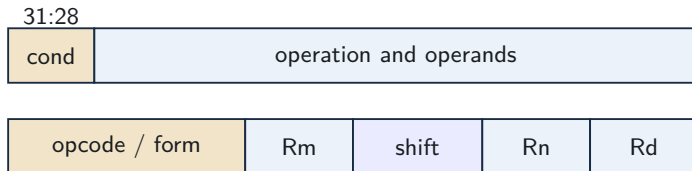
`cmp x0, x1` $\longrightarrow$ `b.lt less` or `csele x2, x3, x4, eq`

Condition	Meaning	Condition	Meaning
eq / ne	equal / not equal	ge / lt	signed $\geq$ / $<$
hs / lo	unsigned $\geq$ / $<$	gt / le	signed $>$ / $\leq$
hi / ls	unsigned $>$ / $\leq$	mi / pl	negative / non-negative

Conditional behavior remains, but the 4-bit condition field is no longer attached to most instructions.

From AArch32 Predication to AArch64 Conditions

AArch32 ARM instructions usually reserved bits 31:28 for a condition field. That made most instructions conditionally executable.



AArch64 uses the 32-bit word for larger register fields
and more specialized instruction formats

AArch64 no longer attaches a 4-bit condition field to most instructions. Conditional behavior is handled with conditional branches and selected conditional operations.

Conditional Examples in AArch64

AArch64 does not have a general `add.lt` form.

Move if less than

```
cmp x0, x1
```

```
csel x2, x3, x2, lt
```

If $x0 < x1$, then $x2$ gets $x3$; otherwise $x2$ keeps its old value.

Add if less than

```
cmp x0, x1
```

```
b.ge skip
```

```
add x2, x2, #1
```

```
skip:
```

The add executes only when $x0 < x1$.

Use conditional selection when choosing between values; use a branch when the arithmetic instruction itself should be conditional.

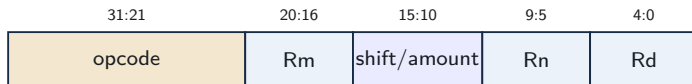
Topic 7

Data-Processing Instruction Encodings

Data Processing: Register Form

Example Instruction

add x5, x6, x7



- Rd is the destination register.
- Rn is the first source register.
- Rm is the second source register.
- Shift fields are available because this form supports shifted register operands.

Encoding Example: add x5, x6, x7

Register Numbers

$Rd = x5 = 00101$

$Rn = x6 = 00110$

$Rm = x7 = 00111$

Operation Fields

operation = add

operand size = 64-bit

shift amount = 0

10001011000	00111	000000	00110	00101
-------------	-------	--------	-------	-------

The complete encoding is

`10001011000 00111 000000 00110 00101 = 0x8B0700C5.`

Data Processing: Immediate Form

Example Instruction

add x5, x6, #24



- The immediate field is 12 bits wide.
- The shift field can optionally shift the immediate by 12 bits.
- This format is common for address arithmetic and small integer constants.

Encoding Example: add x5, x6, #24

Operands

$Rd = x5 = 00101$

$Rn = x6 = 00110$

$imm12 = 24 = 000000011000$

Instruction Meaning

$x5 = x6 + 24$

The immediate value is part of the instruction word.

10010001	00	000000011000	00110	00101
----------	----	--------------	-------	-------

add x5, x6, #24 is encoded as 0x910060C5.

AArch64 Immediate Encodings Are Specialized

- AArch64 does not use the AArch32 rotated-imm8 data-processing scheme.
- Different instruction families have different immediate encodings.
- Some constants fit directly in an add or sub immediate.
- Other constants require move-wide instructions such as movz, movn, or movk.
- Logical instructions use their own bitmask-immediate encoding.

Immediate Fields

Do not ask whether a constant fits “in AArch64” in general. Ask whether it fits in the specific immediate field of the instruction being used.

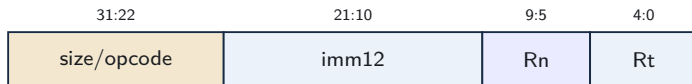
Topic 7

Memory and PC-Relative Encodings

Load/Store Instruction Encoding

Example Instruction

`ldr x5, [x6, #16]`



- Rt is the register loaded or stored.
- Rn is the base address register.
- imm12 is a scaled unsigned offset for this common form.
- For a 64-bit `ldr`: $\text{address} = \text{x6} + 8 \times \text{imm12}$.

Encoding Example: `ldr x5, [x6, #16]`

Operands

$Rt = x5 = 00101$

$Rn = x6 = 00110$

$\#16 = 2 \times 8 \text{ bytes}$

Encoded Offset

$\text{imm12} = 2 = 000000000010$

The instruction stores the scaled value, not the byte offset directly.

1111100101	000000000010	00110	00101
------------	--------------	-------	-------

`ldr x5, [x6, #16]` is encoded as `0xF94008C5`.

Store Uses the Same Address Fields

`str x5, [x6, #16]` is encoded as `0xF90008C5`.

1111100100	000000000010	00110	00101
------------	--------------	-------	-------

- The base register and scaled offset fields match the load example.
- The operation field changes from load to store.
- `Rt` is now the source register whose value is written to memory.

Scaled Offsets Depend on Access Size

Unsigned Offset Form

$$\text{address} = \text{base} + \text{imm12} \times \text{access size}$$

Instruction kind	Access size	Encoded immediate meaning
ldrb/strb	1 byte	byte offset
ldrh/strh	2 bytes	halfword offset divided by 2
ldr w/str w	4 bytes	word offset divided by 4
ldr x/str x	8 bytes	doubleword offset divided by 8

Scaled Offset Range

Scaling lets a 12-bit field cover a larger byte range for naturally aligned accesses.

PC-Relative Addresses in AArch64

- PC-relative instructions encode an offset from the current instruction address.
- This supports position-independent code and nearby constants or labels.
- AArch64 does not use the AArch32 “PC + 8” rule for these examples.

Common PC-Relative Forms

- `adr`: forms an address near the current PC.
- `adrp`: forms a page address relative to the current PC page.
- `ldr literal`: loads data from a nearby PC-relative literal location.
- `b` and `bl`: branch to a PC-relative target.

Literal Loads in AArch64

Concept

A literal load reads a constant stored near the instruction stream using a PC-relative offset.



- The instruction encodes where to find the constant relative to the PC.
- The loaded value is data, even though it may be placed near the code.
- Assemblers may create literal pools automatically when a constant does not fit a simple immediate form.

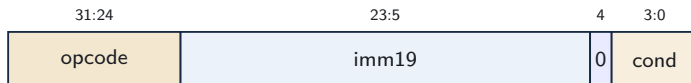
Topic 7

Branch Encodings

Conditional Branch Encoding

Example Forms

`b.eq label` `b.lt label` `b.ne label`



- The condition code is encoded in the low 4 bits.
- The signed immediate is 19 bits, scaled by 4 bytes.
- Conditional branches test the flags set by earlier compare or arithmetic instructions.

Conditional Branch Example: b.eq

Target Offset

current PC = 0x100

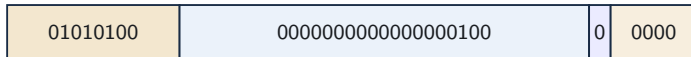
target = 0x110

imm19 = 4

Condition Field

eq = 0000

The branch is taken when the Z flag is set.



b.eq target is encoded as 0x54000080.

Unconditional Branch Encoding

Instruction Forms

b label

bl function



- b and bl use a signed 26-bit immediate field.
- The offset is measured in 4-byte instructions, not raw bytes.
- bl also writes the return address into x30.

Target Calculation

$$\text{target address} = \text{PC} + \text{signextend}(\text{imm26}) \times 4$$

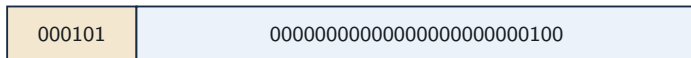
Branch Example: Forward Target

Code Shape

```
0x100:  b target
0x104:  add x0, x0, #1
0x108:  sub x1, x1, #1
0x10c:  nop
0x110:  target:
```

Offset

$$\begin{aligned}\Delta &= 0x110 - 0x100 \\ &= 16 \text{ bytes} \\ \text{imm26} &= 16/4 = 4\end{aligned}$$



b target is encoded as 0x14000004.

Topic 7

Decoding and Reverse Engineering Machine Code

Decoding Checklist

- ① Convert the hexadecimal word to 32 binary bits.
- ② Identify the broad encoding family from the opcode bits.
- ③ Split the word into the fields for that family.
- ④ Translate register fields into x or w register names.
- ⑤ Interpret immediates carefully: sign extension, scaling, and shifts matter.
- ⑥ Reconstruct the assembly instruction.

Important

The same bit positions can mean different things in different instruction families. Always identify the family before naming the fields.

Reverse Engineering Example

Given Machine Code

0x8B0700C5

10001011000	00111	000000	00110	00101
add x-register	Rm = x7	no shift	Rn = x6	Rd = x5

Decoded Instruction

0x8B0700C5 → add x5, x6, x7

Common Encoding Mistakes

- Treating a register number as decimal text instead of a binary field.
- Forgetting that AArch64 register fields are usually 5 bits.
- Encoding a byte offset directly when the immediate field is scaled.
- Applying the AArch32 $PC + 8$ rule to AArch64 branch examples.
- Reading little-endian memory bytes as if they were the printed instruction word.

Practical Habit

Write the bit ranges first, then fill in the fields. Do not try to guess the final hexadecimal word too early.

Topic 7: Summary

- AArch64 instructions are fixed-width 32-bit words.
- The meaning of each bit field depends on the instruction family.
- Data-processing register instructions encode register operands and optional shift information.
- Data-processing immediate instructions use a different field layout.
- Load/store instructions commonly encode a base register, target/source register, and scaled offset.
- Branch instructions encode PC-relative offsets scaled by 4 bytes.
- AArch64 uses conditional branches and conditional operations rather than general per-instruction predication.

Self-Test Questions: Instruction Words

- ❶ Why does AArch64 use 5-bit register fields?
- ❷ What is the difference between the printed hexadecimal instruction word and the byte order in memory?
- ❸ Why must the instruction family be identified before interpreting the fields?
- ❹ What fields are needed to encode `add x5, x6, x7`?
- ❺ Why does `add x5, x6, #24` use a different format from `add x5, x6, x7`?

Self-Test Questions: Memory and Branches

- ❶ In `ldr x5, [x6, #16]`, why is the encoded immediate 2 rather than 16?
- ❷ What is the role of `Rt` in a load? What is its role in a store?
- ❸ How is the target of an unconditional AArch64 branch computed?
- ❹ How does AArch64 conditional execution differ from AArch32 predication?
- ❺ Where is the condition code stored in a conditional branch instruction?
- ❻ Decode `0x8B0700C5` back into the corresponding AArch64 assembly instruction.

Looking Ahead

Next Topic

Building a simple single-cycle RISC datapath.

- Instruction fields become inputs to the datapath and control logic.
- Register fields select operands in the register file.
- Immediate fields must be extracted, extended, and routed.
- Branch offsets feed the next-PC calculation.